

Python Programming

Practice 8 — Week 8: Modules, Packages, and Unit Tests
Academic Year 2025–2026

Topics Covered

Task	Topic	Concepts
1	Modules	Split classes into separate .py files, import between files
2	Packages	Create analytics/ package with __init__.py, dot imports
3	Unit Tests	unittest.TestCase, setUp(), assert methods, test cases
4	Run Tests	python -m unittest -v, reading test output

Overview

This practice builds on Practice 7. You will refactor your existing class structure into separate module files, organise them into a package, and write unit tests to verify that your analyser works correctly.

Final Project Structure

```
project/
├── main.py
├── students.csv
├── analytics/
│   ├── __init__.py
│   ├── file_manager.py
│   ├── data_loader.py
│   ├── analyser.py
│   ├── result_saver.py
│   └── report.py
├── tests/
│   ├── __init__.py
│   └── test_analyser.py
```

Submission & Defense

Files to submit	All .py files + folder structure (zip or GitHub repo)
Platform	LMS — lms.astanait.edu.kz
Deadline	Before Week 9 practice session
Defense	Week 8 — bring your laptop. Instructor removes key parts; you have 5 minutes to restore the logic.

■ Late submissions are not accepted. All 4 tests in Task 4 must pass.

Tasks 1 & 2

Task 1 — Split Code into Modules (All Variants)

Refactor your Practice 7 code so that each class lives in its own .py file. main.py imports from all modules and runs the program.

Technical requirements:

- ✓ Create the following files in your project folder:
 - file_manager.py — contains class FileManager
 - data_loader.py — contains class DataLoader
 - analyser.py — contains class DataAnalyser and your child class
 - result_saver.py — contains class ResultSaver
 - report.py — contains class Report
 - main.py — imports from all modules and runs the program
- ✓ Each file must have its own imports at the top — only import what that file needs
- ✓ main.py must not contain any class definitions — only imports and calls

Expected imports in main.py:

```
from file_manager import FileManager
from data_loader import DataLoader
from analyser import GpaAnalyser    # use your variant's class
from result_saver import ResultSaver
from report import Report
```

The console output must be identical to Practice 7 — splitting into modules changes the structure, not the behaviour.

Task 2 — Create a Package (All Variants)

Turn the module files into a proper Python package by placing them inside a folder with an __init__.py file.

Technical requirements:

- ✓ Create a subfolder called analytics/
- ✓ Move all module files into analytics/:
 - file_manager.py, data_loader.py, analyser.py, result_saver.py, report.py
- ✓ Create analytics/__init__.py and import all classes inside it:

```
from .file_manager import FileManager
from .data_loader import DataLoader
from .analyser import DataAnalyser
from .result_saver import ResultSaver
from .report import Report
```

- ✓ Update main.py to import from the package:

```
from analytics import FileManager, DataLoader, ResultSaver, Report
from analytics.analyser import GpaAnalyser    # your variant's class
```

- ✓ The program must run correctly after the restructure — same output as before

Hint: the dot in from .file_manager import FileManager means 'from the same package' — this is a relative import.

Task 3 — Write Unit Tests (All Variants)

Create a test package and write 4 unit tests for your analyser class. Tests use a small hardcoded dataset — do not load the CSV file inside tests.

Setup:

- ✓ Create a subfolder called tests/
- ✓ Create tests/__init__.py — leave it empty
- ✓ Create tests/test_analyser.py
- ✓ Import unittest and your analyser class at the top:

```
import unittest
from analytics.analyser import GpaAnalyser    # use your variant's class
```

setUp() method:

Define a setUp(self) method inside your test class. It runs automatically before each test and creates a small 5-row sample dataset:

```
def setUp(self):
    self.sample = [
        {"GPA": "3.8", "sleep_hours": "7", "country": "USA",
         "final_exam_score": "95", "study_hours_per_day": "4"},
        {"GPA": "2.5", "sleep_hours": "5", "country": "India",
         "final_exam_score": "72", "study_hours_per_day": "2"},
        {"GPA": "3.9", "sleep_hours": "8", "country": "USA",
         "final_exam_score": "98", "study_hours_per_day": "5"},
        {"GPA": "1.8", "sleep_hours": "4", "country": "Canada",
         "final_exam_score": "55", "study_hours_per_day": "1"},
        {"GPA": "3.5", "sleep_hours": "6", "country": "India",
         "final_exam_score": "88", "study_hours_per_day": "3"},
    ]
```

Test 1 — test_result_is_not_empty:

After calling analyse(), self.result must not be an empty dictionary.

```
def test_result_is_not_empty(self):
    analyser = GpaAnalyser(self.sample)
    analyser.analyse()
    self.assertNotEqual(analyser.result, {})
```

Test 2 — test_total_students:

The result must contain the correct total student count (5).

```
def test_total_students(self):
    analyser = GpaAnalyser(self.sample)
    analyser.analyse()
    self.assertEqual(analyser.result["total_students"], 5)
```

Test 3 — test_result_has_required_keys:

The result dictionary must contain all expected keys for your variant.

Variant	Required keys in result
A	"average_gpa", "max_gpa", "min_gpa", "high_performers"
B	"total_countries", "top_3", "all_countries"
C	"low_sleep", "high_sleep", "gpa_difference"

D	"top_10"
---	----------

```
def test_result_has_required_keys(self):
    analyser = GpaAnalyser(self.sample)
    analyser.analyse()
    self.assertIn("average_gpa", analyser.result)
    self.assertIn("max_gpa", analyser.result)
    self.assertIn("min_gpa", analyser.result)
    self.assertIn("high_performers", analyser.result)
```

Test 4 — test_analyse_twice:

Calling analyse() twice must produce the same result both times.

```
def test_analyse_twice(self):
    analyser = GpaAnalyser(self.sample)
    analyser.analyse()
    result1 = analyser.result.copy()
    analyser.analyse()
    self.assertEqual(analyser.result, result1)
```

Task 4 — Run the Tests (All Variants)

Run your unit tests from the terminal and paste the output as a comment at the top of `test_analyser.py`.

Technical requirements:

- ✓ Run the tests from your project root folder using:

```
python -m unittest tests/test_analyser.py -v
```

- ✓ All 4 tests must pass — fix your analyser code if any test fails
- ✓ Copy the terminal output and paste it as a comment at the top of `test_analyser.py`:

```
# Test results:
# test_analyse_twice (tests.test_analyser.TestAnalyser) ... ok
# test_result_has_required_keys (tests.test_analyser.TestAnalyser) ... ok
# test_result_is_not_empty (tests.test_analyser.TestAnalyser) ... ok
# test_total_students (tests.test_analyser.TestAnalyser) ... ok
#
# Ran 4 tests in 0.002s
#
# OK
```

Full test file structure:

```
# Test results:
# ... (paste terminal output here)

import unittest
from analytics.analyser import GpaAnalyser

class TestAnalyser(unittest.TestCase):

    def setUp(self):
        self.sample = [ ... ]

    def test_result_is_not_empty(self):
        ...

    def test_total_students(self):
        ...

    def test_result_has_required_keys(self):
        ...

    def test_analyse_twice(self):
        ...

if __name__ == '__main__':
    unittest.main()
```

Common assert methods:

Method	What it checks
<code>assertEqual(a, b)</code>	<code>a == b</code>
<code>assertNotEqual(a, b)</code>	<code>a != b</code>

<code>assertIn(key, dict)</code>	key exists in dict
<code>assertTrue(expr)</code>	expr is True
<code>assertGreater(a, b)</code>	$a > b$
<code>assertIsNotNone(obj)</code>	obj is not None